

LLM Proxy — Development Design Document

Project: DigitalOceanTest / llm-proxy

Stack: Java 21, Spring Boot 3.4

Author: Engineering Team

Status: Implemented

Version: 1.0

1. Problem Statement

Teams operating LLM-backed APIs need a safe way to evaluate alternative models (candidate) without risking customer experience on the production model (primary). Directly routing customer traffic to an unproven model introduces latency, quality, and availability risk.

We require a production-ready proxy that serves all customer traffic through a trusted primary DigitalOcean Serverless Inference endpoint while asynchronously mirroring the same requests to a candidate model, comparing outputs with deterministic rules, and exposing real-time observability so operators can decide when a candidate is ready for promotion.

2. Goals

- Expose POST /v1/chat that forwards payloads to primary DO Serverless Inference and returns immediately.
- Asynchronously route identical payloads to a candidate model on a decoupled background thread pool.
- Ensure candidate latency, errors, timeouts, or queue saturation never delay or alter the primary HTTP response.
- Compare primary vs candidate outputs using deterministic heuristics (valid JSON + exact action match).
- Expose GET /metrics with total requests, shadow errors/timeouts, dropped shadow tasks, and exact match rate.
- Bound memory footprint under burst via bounded thread pool, bounded queue, load shedding, and capped comparison history.
- Provide mock mode for local development and CI without live API keys.
- Run automated tests and GitHub Actions CI on every push.

3. Non-Goals

- Streaming chat completions (explicitly rejected with HTTP 400).
- Semantic similarity or LLM-as-judge comparison (non-deterministic scoring).
- Persistent trace storage or SQLite mismatch audit trail (discussed as future work).
- Runtime PUT /config shadow mirror percentage throttle (discussed as future work).
- Customer-facing exposure of candidate responses or A/B routing at request time.
- Multi-region deployment, autoscaling, or Kubernetes manifests in v1.
- Authentication/authorization on public endpoints in v1 (recommended before production).

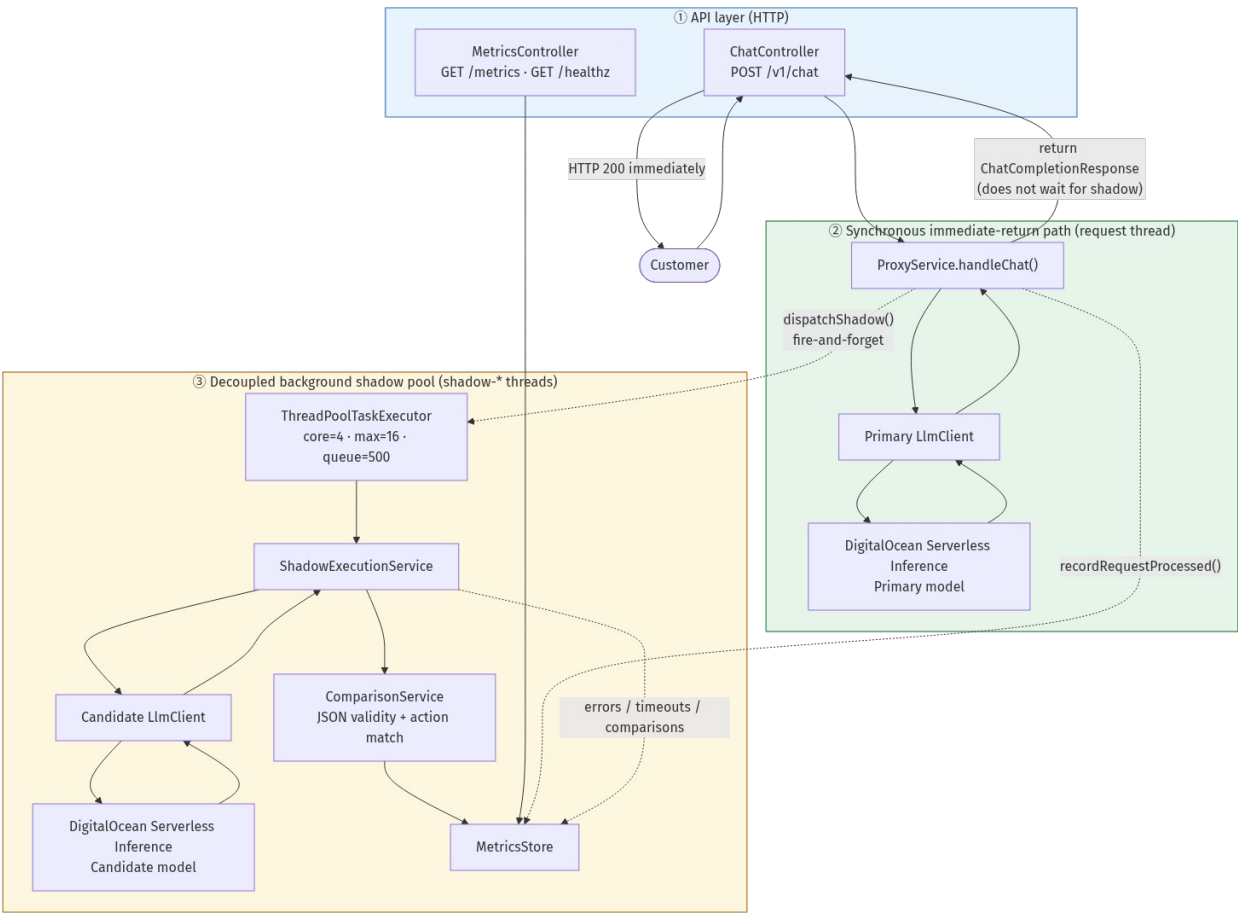
4. Approach

We implemented a three-layer Spring Boot service: (1) HTTP API layer, (2) synchronous primary path on the request thread, and (3) decoupled shadow evaluation pool. The primary path is strictly sequential: validate → call primary LlmClient → return ChatCompletionResponse. Shadow work is fire-and-forget via BoundedShadowExecutor after primary success.

Comparison runs only after candidate response arrives. Metrics are aggregated in-memory with atomic counters and a ring-buffer deque for recent comparison records. Under overload, shadow tasks are dropped (never queued unboundedly, never executed on HTTP threads).

5. Architecture

5.1 Layer Map



Layer	Components	Thread	Customer Impact
API	ChatController, MetricsController	HTTP worker	Entry/exit for all traffic

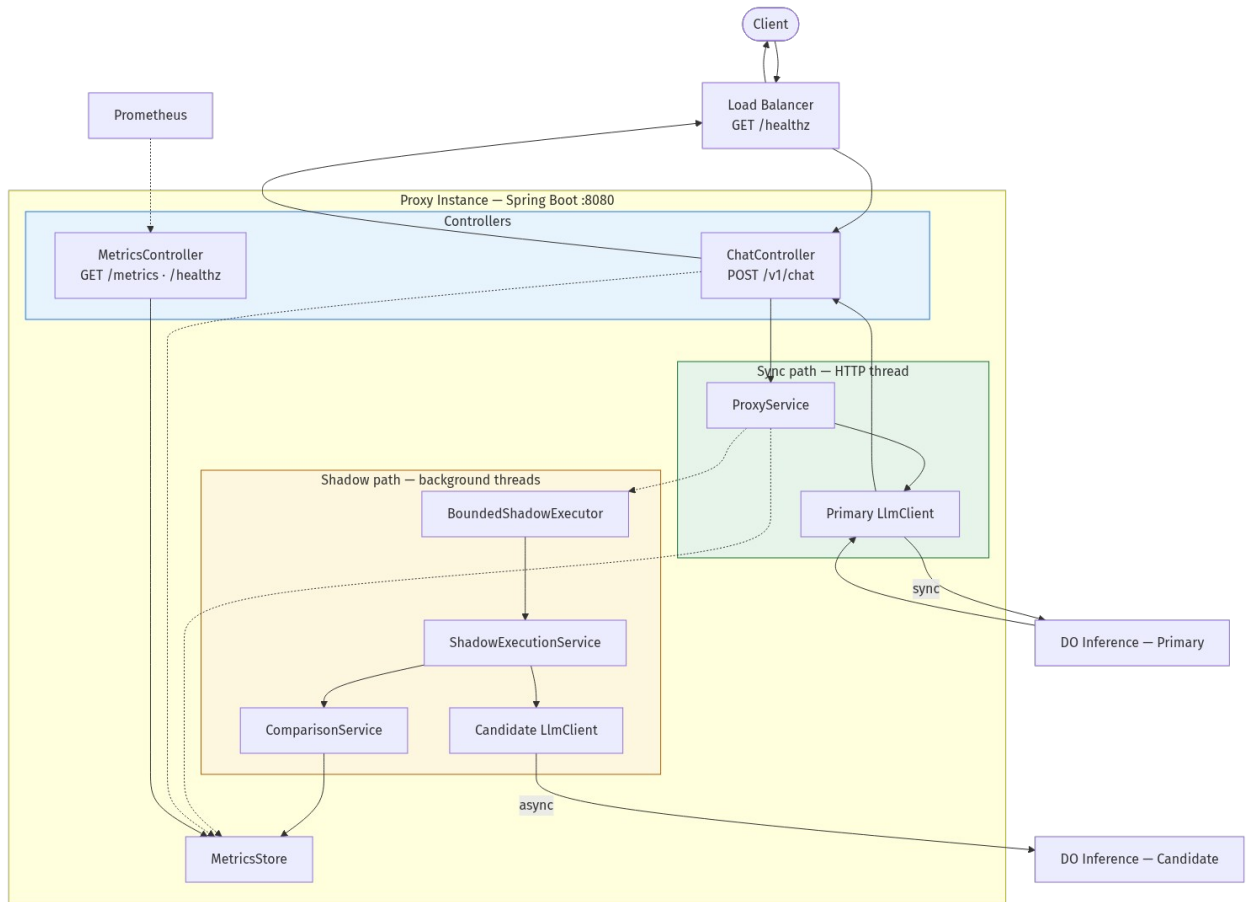
Sync path	ProxyService → primary LlmClient → DO Inference	HTTP thread	Blocks only on primary
Shadow pool	BoundedShadowExecutor → ShadowExecutionService → candidate → ComparisonService	shadow-* threads	Fully decoupled; failures invisible to customer

5.2 Component Responsibilities

Component	Responsibility
ChatController	POST /v1/chat — counts requests, delegates to ProxyService
MetricsController	GET /metrics, GET /healthz
ProxyService	Primary orchestration; schedules shadow after primary success
LlmClient (primary/candidate)	HTTP to DO /v1/chat/completions; mock or live mode
BoundedShadowExecutor	Bounded thread pool + queue; load shedding
ShadowExecutionService	Candidate call, comparison, metrics recording
ComparisonService	JSON validity + action field exact match
MetricsStore	Counters, ring buffer, ObservabilitySummary
AppProperties	Configuration from application.yml / .env

5.3 Infrastructure & Request Flow

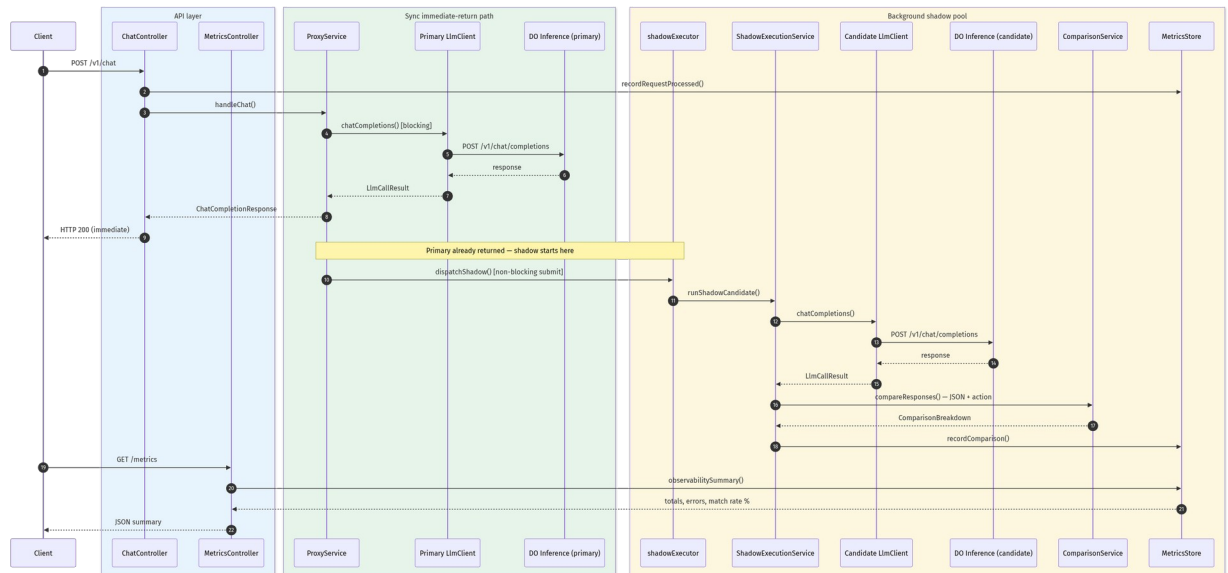
In production, clients connect through a load balancer to one of N identical Spring Boot replicas (Docker container, Droplet, or Kubernetes pod). All controllers and services listed above run inside every replica — ProxyService, ShadowExecutionService, and ComparisonService are not separate deployed microservices. The load balancer health-checks GET /healthz and round-robins POST /v1/chat to any healthy instance.



Step	Path	Components
1	Edge	Client → Load Balancer (TLS, /healthz health check)
2	API	LB → ChatController → ProxyService
3	Sync	ProxyService → Primary LlmClient → DO primary → HTTP 200 to client
4	Shadow	ProxyService → BoundedShadowExecutor → ShadowExecutionService → Candidate LlmClient
5	Compare	ComparisonService → MetricsStore
6	Ops	MetricsController / Prometheus scrape → MetricsStore
Component		Source file

ChatController	controller/ChatController.java
MetricsController	controller/MetricsController.java
ProxyService	service/ProxyService.java
ShadowExecutionService	service/ ShadowExecutionService.java
ComparisonService	service/ComparisonService.java
MetricsStore	service/MetricsStore.java
BoundedShadowExecutor	service/ BoundedShadowExecutor.java
LlmClient (primary + candidate)	client/LlmClient.java

6. Sequence Diagram



Key invariant: steps 1-8 (primary path) complete before the customer receives HTTP 200. Shadow steps 9-16 run concurrently and never block step 8.

7. API Design

Method	Path	Description
POST	/v1/chat	OpenAI-compatible chat proxy (primary path)
GET	/metrics	Real-time observability JSON summary
GET	/healthz	Liveness probe
GET	/actuator/prometheus	Optional Prometheus scrape

7.1 Metrics Response Fields

Field	Meaning
-------	---------

total_requests_processed	All /v1/chat requests received
shadow_errors_or_timeouts	Candidate failures (customer unaffected)
shadow_tasks_dropped	Shadow evaluations shed when pool saturated
exact_match_rate_percent	% comparisons with matching action field
total_comparisons	Completed shadow evaluations
action_exact_matches	Count of exact action matches
pending_shadow_executions	Shadow jobs in flight

8. Comparison Heuristics

- Parse primary and candidate message content as JSON objects (supports markdown ```json fences).
- Extract the action string field from each payload.
- action_exact_match = both valid JSON AND primary.action equals candidate.action (exact string equality).
- exact_match_rate_percent = action_exact_matches / total_comparisons × 100.

9. Data Structures & Algorithms

Structure / Algorithm	Usage	Complexity / Bound
ThreadPoolExecutor + ArrayBlockingQueue	Shadow task scheduling	O(1) submit; bounded queue
ArrayDeque ring buffer	Comparison history in MetricsStore	O(1) append; evict oldest when > max
AtomicLong / AtomicInteger	Hot counters (requests, drops, pending)	Lock-free increments
Jackson readTree + regex fence strip	JSON parsing in ComparisonService	O(n) on payload size
SHA-256 (mock mode)	Deterministic mock LLM responses	O(n) on input length
Deep copy via Jackson convertValue	Isolate shadow request copy	O(n); only on shadow thread

10. Configuration

Variable	Default	Purpose
MOCK_LLM	true (tests)	Local mocks vs live DO Inference
LLM_TIMEOUT_SECONDS	60	Upstream DO HTTP response timeout
SHADOW_CORE_POOL_SIZE	4	Minimum shadow worker threads
SHADOW_MAX_POOL_SIZE	16	Maximum parallel

SIZE		shadow executions
SHADOW_QUEUE_CAPACITY	500	Max queued shadow tasks before shedding
MAX_COMPARISON_RECORDS	1000	Ring buffer size for comparison history
PRIMARY_LLM_MODEL	openai-gpt-oss-20b	Primary model ID
CANDIDATE_LLM_MODEL	deepseek-4-flash	Candidate model ID

11. Trade-offs & Design Decisions

Decision	Chosen	Alternative	Rationale
Shadow start timing	After primary returns	Parallel with primary	Avoid 2× API cost on primary failure; simpler lifecycle
Overload policy	Drop shadow tasks	Block or CallerRuns	Never impact customer latency or HTTP threads
Comparison method	Deterministic JSON/action	Semantic / LLM judge	Reproducible, testable, explainable
Metrics storage	In-memory aggregates	Database	Fast, simple; sufficient for live SLA view
Language	Java 21 / Spring Boot	Python FastAPI (initial)	Production thread pools, typing, ecosystem
Primary/candidate clients	Two @Qualifier beans	Single client with param	Clear separation, independent config
Mock mode	Built into LlmClient	External stub server	Zero-deps local dev and CI

12. Memory Footprint Under Load

Worst-case shadow memory is approximately $O(\text{SHADOW_QUEUE_CAPACITY} + \text{SHADOW_MAX_POOL_SIZE} + \text{MAX_COMPARISON_RECORDS})$. Primary footprint scales with concurrent HTTP connections only — not with shadow backlog.

- Pre-check capacity before shadow submit; reject handler as safety net.
- Lazy request deep-copy inside shadow runnable (shed tasks allocate nothing).
- DiscardPolicy semantics — dropped tasks are not retained in memory.

- Comparison deque evicts oldest entries beyond MAX_COMPARISON_RECORDS.

13. Testing & CI

- Unit: BoundedShadowExecutorTest, MetricsStoreTest, ComparisonServiceTest, ShadowExecutionServiceTest
- Integration: ProxyIntegrationTest, ShadowLoadSheddingIntegrationTest (20-request burst)
- GitHub Actions: Java 21, ./mvnw test, MOCK_LLM=true on push/PR to main

14. Future Enhancements

- PUT /config for runtime shadow_mirror_percent (0-100) throttle.
- SQLite persistent traces for mismatched payloads (debug / visualization).
- Parallel primary + candidate start for faster comparisons (at higher API cost).
- Auth on admin endpoints; rate limiting on /v1/chat.
- Streaming support with shadow on completed non-stream responses.

15. References

- DigitalOcean Serverless Inference: <https://cloud.digitalocean.com/model-studio/serverless-inference>
- DO Inference API: <https://docs.digitalocean.com/products/inference/reference/api/serverless-inference/>
- Repository README and docs/diagrams/ in DigitalOceanTest project